Security Assurance for Modern Big Data APIs

Enea Manzi[1], Marco Anisetti[1,2]

enea.manzi@unimi.it
https://eneamanzi.github.io/

[1]Department of Computer Science, Università degli Studi di Milano

[2]Moon Cloud srl

ITADATA 2026

Context: Big Data APIs & Security Assurance

- **Big Data Ecosystem:** distributed services exposed through APIs and protected by API gateways

Context: Big Data APIs & Security Assurance

- **Big Data Ecosystem:** distributed services exposed through APIs and protected by API gateways
- **Trust Boundary:** APIs and gateways control access to pipelines, data, and services

Context: Big Data APIs & Security Assurance

- **Big Data Ecosystem:** distributed services exposed through APIs and protected by API gateways
- **Trust Boundary:** APIs and gateways control access to pipelines, data, and services
- **Assurance Gap:** syntactic and functional checks do not prove the expected security behavior

Examples of behavior violations

User A requests User B's resource	$\rightarrow$	unauthorized access
Expired or revoked credential	$\rightarrow$	request accepted

State of the Art Limitations and Objectives

State of the Art Limitations

- **Syntactic Blindness:**
  valid requests can still violate security

Our Objectives

- **Semantic Verification:**
  check security behavior, not only syntax

State of the Art Limitations and Objectives

State of the Art Limitations

- **Syntactic Blindness:**
  valid requests can still violate security
- **The Oracle Problem:**
  no criterion to judge a violation

Our Objectives

- **Semantic Verification:**
  check security behavior, not only syntax
- **Specified Oracles:**
  an explicit rule for each guarantee

State of the Art Limitations and Objectives

State of the Art Limitations

- **Syntactic Blindness:**
  valid requests can still violate security
- **The Oracle Problem:**
  no criterion to judge a violation
- **Shallow & Flaky Tools:**
  tools trade depth for reach, results vary across runs

Our Objectives

- **Semantic Verification:**
  check security behavior, not only syntax
- **Specified Oracles:**
  an explicit rule for each guarantee
- **Agnostic & Deterministic:**
  portable across targets, reproducible across runs

Our Approach: Overview

Core Idea: Derive valid tests from OpenAPI contracts and evaluate security behavior using specified, deterministic oracles.

- **Application Agnosticism & Contract-Driven Testing**
- **Eight-Domain Taxonomy & Privilege-Aware Coverage**
- **Specified Oracles for Security Guarantees**
- **Deterministic Assessment Engine**

Our Approach: Agnosticism and Contract-Driven

- **Application agnosticism:** no target-specific code; the target is described by its OpenAPI spec and configuration
- **Configuration as source of variation:** all target-specific values live in one file
- **Contract-driven testing:** the spec defines the attack surface and guides the construction of valid requests

Our Approach: Taxonomy and Privilege-Aware Coverage

8-Domain Taxonomy

ID	Domain
D0	API Discovery
D1	Identity & Authentication
D2	Authorization
D3	Data Integrity
D4	Availability & Resilience
D5	Observability
D6	Configuration & Hardening
D7	Business Logic & Sensitive Flows

Our Approach: Taxonomy and Privilege-Aware Coverage

8-Domain Taxonomy

ID	Domain
D0	API Discovery
D1	Identity & Authentication
D2	Authorization
D3	Data Integrity
D4	Availability & Resilience
D5	Observability
D6	Configuration & Hardening
D7	Business Logic & Sensitive Flows

Privilege-Aware Box Gradient

- **Black:** *D1: does the service reject an expired or revoked credential?*
- **Grey:** *D2: can user A read the resources of user B?*
- **White:** *D4: does the gateway enforce the rate limit?*

Our Approach: Specified Oracles

Every test has its own specified oracle, built in three steps:

1. Identify the guarantee $\rightarrow$ **2.** Define the violation $\rightarrow$ **3.** Encode the rule

Our Approach: Specified Oracles

Every test has its own specified oracle, built in three steps:

1. Identify the guarantee → **2.** Define the violation → **3.** Encode the rule

Domain	Criterion from	Violation
D1 Authentication	credential rules	expired or revoked credential accepted
D2 Authorization	two roles compared	user A reads user B's resource
D4 Availability	gateway configuration	rate limit not enforced

Our Approach: Deterministic Assessment Engine

- **Dependency-aware scheduling:** a test runs only after its prerequisites
- **Reproducible assessment:** fixed configuration, order and oracles $\rightarrow$ same configuration, same verdict
- **Actionable output:** distinct exit codes separate a security violation from an infrastructure error

Experimental Setup

Target and gateway

- **Forgejo** 14.0.3 – **Kong** 3.9 DB-less
- PostgreSQL 15
- 3 accounts: admin + 2 users

Interface properties

shared by Big Data services behind a gateway

- Native OpenAPI specification
- Distinct privilege levels and parameterized endpoints
- Gateway configuration available

Experimental Setup

Target and gateway

- **Forgejo** 14.0.3 – **Kong** 3.9 DB-less
- PostgreSQL 15
- 3 accounts: admin + 2 users

Interface properties

shared by Big Data services behind a gateway

- Native OpenAPI specification
- Distinct privilege levels and parameterized endpoints
- Gateway configuration available

The testbed combines:

- **Expected success:** e.g. security headers added by the gateway
- **Deliberately broken:** e.g. rate limiting disabled
- **Native behavior:** other outcomes come from Forgejo and Kong as deployed

Experimental Results

Expected testbed outcomes correctly classified

9 PASS • 7 FAIL • 2 SKIP • 0 ERROR

18 tests in 7 domains | 2 independent runs | identical outcomes

Experimental Results

Expected testbed outcomes correctly classified

9 PASS • 7 FAIL • 2 SKIP • 0 ERROR

18 tests in 7 domains | 2 independent runs | identical outcomes

Key result: Contract vs. Behavior (D1)

Spec: endpoints declared **protected**
Observed: 2xx without credentials
Verdict: BYPASSED (74 findings)

Public by design: the contract is wrong $\rightarrow$ an analyst decides

Conclusions & Future Work

Contract-driven security assurance for Big Data APIs, with specified oracles

- Verifies security behavior, not only syntax
- Same outcomes across independent runs $\rightarrow$ ready for CI/CD
- Found a real contract–behavior discrepancy on Forgejo

Conclusions & Future Work

Contract-driven security assurance for Big Data APIs, with specified oracles

- Verifies security behavior, not only syntax
- Same outcomes across independent runs $\rightarrow$ ready for CI/CD
- Found a real contract–behavior discrepancy on Forgejo

Future Work

- Automatic generation of security oracles
- Complete the coverage of the taxonomy
- Validation on other targets

Thanks! Questions?

Enea Manzi
enea.manzi@unimi.it
https://eneamanzi.github.io/
https://www.linkedin.com/in/eneamanzi